CP 330 – Edge AI, Final Project

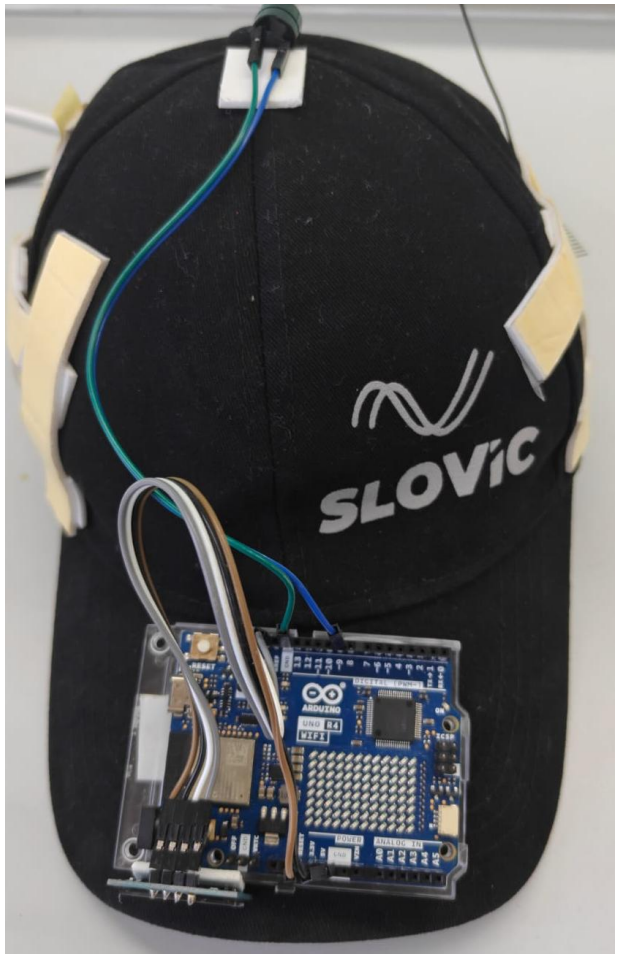
Human Activity Recognition System Using Head-Mounted IMU Sensors

By –
Maitreyi Tiwari, 26500
Tishha Agrawal, 26023
Parthib Kumar Dey, 26787
Abha Singh Sardar, 27086

GitHub link:
abhaanisha/Head-Gesture-Recognition-System

Agenda

- Problem Statement
- Prototyping and Device Setup
- Data Collection
- Model Development
- Deployment
- Conclusion & Future Scope



Problem Statement

Many elderly and differently-abled individuals who have been prey to conditions like ALS, post-stroke paralysis, and spinal cord injuries have severely limited hand mobility but retain **voluntary head movement.**

This project builds a wearable assistive communication device that:

1. Recognizes **6 distinct head gestures** (+ an idle state)
2. Translates them into messages displayed on an **OLED screen**
3. Triggers **audible buzzer alerts** for each gesture type
4. Runs entirely **on-device**
5. A **website** and **mobile** application has been setup to notify the caregiver in case of an emergency

Prototyping & Device Setup

- **Dual-Sensor Headgear:** We used two **Arduino Nicla Vision** boards mounted on a headband at the temples to capture high-resolution motion data from both sides of the head.
- **Master-Slave Sync:** Prior encountering hardware issues with wired connections, we used **UDP over WiFi** to send 6-axis motion data from the "Slave" board to the "Master" board in real-time.
- **User Feedback:** The prototype includes an **OLED screen** for visual messages and a **buzzer** for audible alerts, all powered by a small power source for true portability.

Data Collection

- **Multi-User Dataset:** We recorded over **20,000** motion samples per gesture from multiple participants to ensure the model could recognize different movement styles and speeds.
- **7-Gesture Protocol:** Data was collected for seven distinct activities (Nod, Shake, Tilt L/R, Look Up/Down, and Idle) to simulate real-world usage.
- **Challenges** -

    1. Wireless synchronization, using UDP over WiFi introduced "network jitter," where data packets from the Slave board arrived at slightly different times, making it difficult to perfectly align the 12-axis signals.

    2. Device thermal heating from continuous WiFi usage leading to unstable connections between devices (master and slave).

Model Development

Exploring Diverse Modeling Approaches

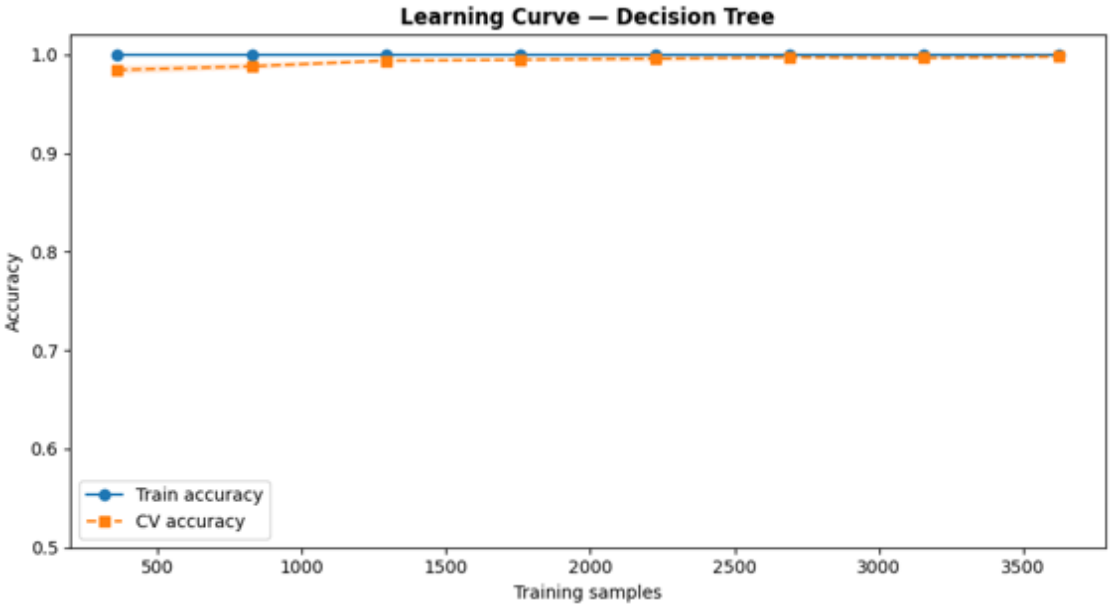
Goal: To identify a model that balances high gesture recognition accuracy with the strict memory (1MB) and power constraints of the Nicla Vision.

Traditional ML (Baseline): Evaluated **Decision Tree** and **Random Forest** classifiers. These models are exceptionally lightweight and provide high "interpretability.“ Also with respect to IMU data these models are known to provide an exceptionally high accuracy.

Deep Learning (Proposed): Developed a **1D-Convolutional Neural Network (CNN)**. Unlike traditional models, a CNN can automatically learn "temporal features" (how a movement changes over time) directly from the 12-axis IMU data.

Model Development

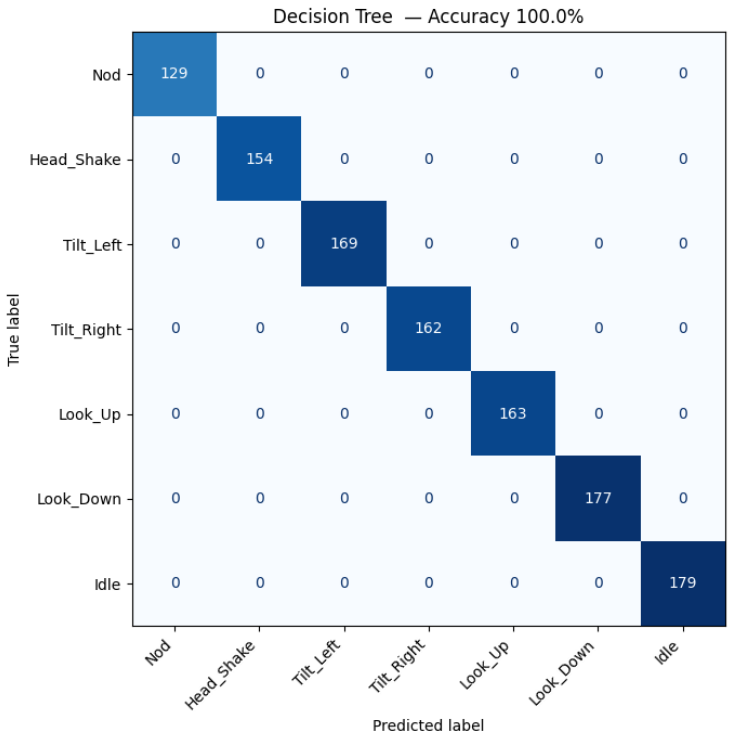
Decision Tree Model



1. Before model development, a sliding window with window size of 50 samples per window and 0.5 overlap was created.
2. Post this we did features extraction comprising of per channel statistical features like mean standard deviation skew etc., spectral features like dominant frequency and entropy and cross IMU features like difference between two IMU signals
3. The decision tree model obtained an accuracy of 100 percent on both the test and train data.
4. The highly accurate model was then exported using m2gen.

Additionally, Random Forest and Keras TinyMLP were also developed (The two of them, however, have not been deployed)

Evaluation Metrics for the decision tree model

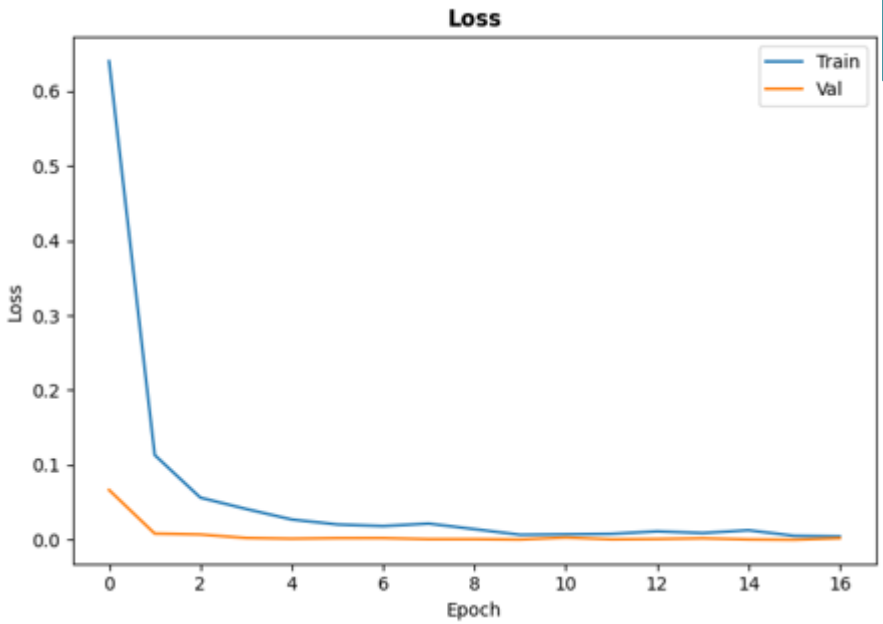



```
Decision Tree Accuracy : 100.00%

Classification Report:
              precision    recall  f1-score   support

         Nod       1.00      1.00      1.00       129
  Head_Shake       1.00      1.00      1.00       154
   Tilt_Left       1.00      1.00      1.00       169
  Tilt_Right       1.00      1.00      1.00       162
     Look_Up       1.00      1.00      1.00       163
   Look_Down       1.00      1.00      1.00       177
        Idle       1.00      1.00      1.00       179

    accuracy                           1.00      1133
   macro avg       1.00      1.00      1.00      1133
weighted avg       1.00      1.00      1.00      1133
```



Model Development

Advanced Feature Extraction with 1D-CNN

Motive: Traditional models look at snapshots; our 1D-CNN looks at **temporal patterns** (how the signal flows over 1.6 seconds – sliding window of size 80).

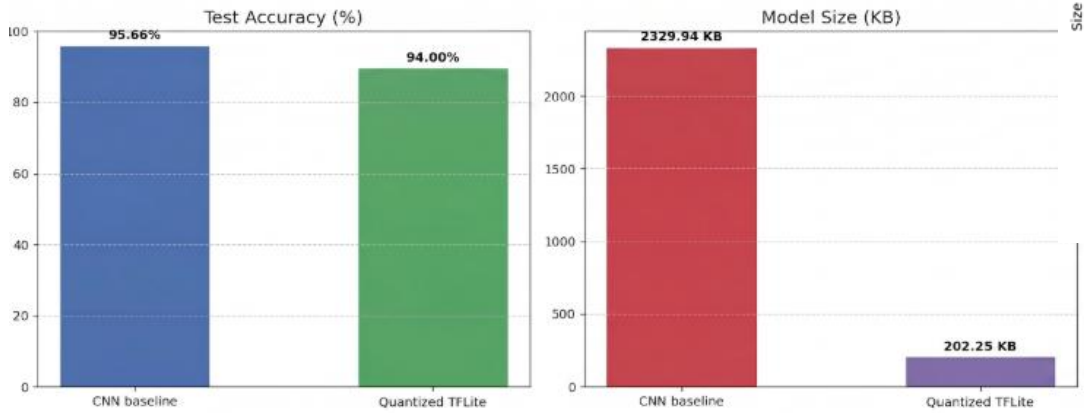
Key Architecture:

- **File-Wise splitting:** Instead of direct concatenation of data to ensure no data contamination.
- **Gaussian Noise Layer:** Acts as "Data Augmentation" to make the model robust against different head movement speeds.
- **Convolutional Filters:** Automatically identify "edges" in the accelerometer data (start of a nod) and "peaks" in the gyroscope data.

Expected Result: A generalized model that focuses on the **shape** of the movement rather than just the raw numbers

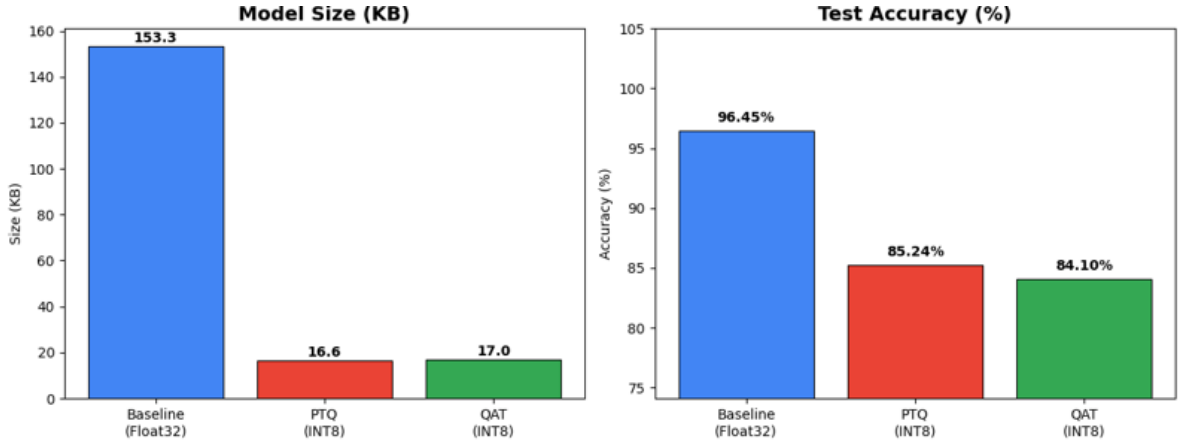
Bottlenecks encountered

- We started with a heavy CNN model with multiple layers and filters to capture the temporal feature space a data in the most efficient manner. Data augmentation was also done to utilize the over parameterization of CNN in the best possible manner and to prevent overfitting over a conventionally smaller data set.
- This baseline model gave an accuracy of 96% with pruned and quantized model getting an accuracy of 94%
- However, because this led to creation of a CNN model with size around 2 MB which after quantization was reduced to around 200 KB. The reduction regardless of being massive did not prove to be ideal for an edge device.



Heavy CNN

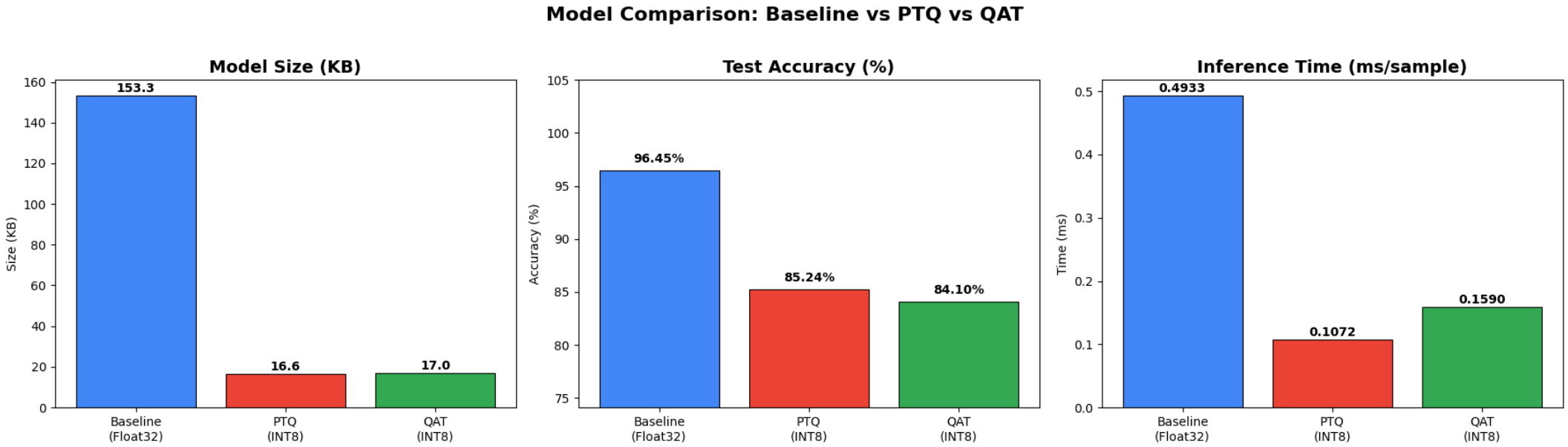
- To overcome this a CNN model with only 4 layers was adopted . While this model achieved a much lower model size of only 15 KB after pruning and quantization The accuracy also drop significantly because we have moved from the overly parameterized architecture to a simpler one which leads to a massive dip in accuracy.



Lightweight CNN

Model Development

Results



Approximately 10x Smaller Footprint: Model size reduced from 153KB to 16KB. via INT8 quantization.
2.4x Faster Inference: Latency dropped to 0.1ms, for post training quantization

Final Model Selection

Model Approach	Model Size	Inference Latency	Accuracy	Memory Type
1D-CNN (Float32)	153.3 KB	0.4933 ms	96.45 %	Needs TFLite RAM
1D-CNN (INT8) (QAT)	17 KB	0.1590 ms	84.10 %	Needs TFLite RAM
1D-CNN (INT8) (PQT)	16.6 KB	0.1072 ms	85.24 %	Needs TFLite RAM
Decision Tree (Deployed)	**1.52 KB**	**< 0.01 ms**	**100%**	**Static Flash**

Based on analyzing multiple metrics like accuracy, model size and ease of deployment on edge devices, Decision tree model came out to be the **undisputed** winner and thus only the decision tree model was deployed on the Nicla Vision for inferencing and for presentation purposes.

Model Deployment

On-Device Inference: The optimized Decision Tree model runs locally on the **Nicla Vision Master**, alongside collection of data on master and receiving data from the slave. ensuring immediate gesture detection without cloud dependency.

As already mentioned in the prototyping section, an OLED screen and a buzzer was used to notify the caregiver in case of an emergency. Both the devices are mounted on headgear as of now however they can be to be used in a much more scalable manner.

Wireless Data Bridge: Detected gestures and raw IMU streams are transmitted via **UDP/WiFi** to a central monitoring station, maintaining high-speed updates.

Model Deployment

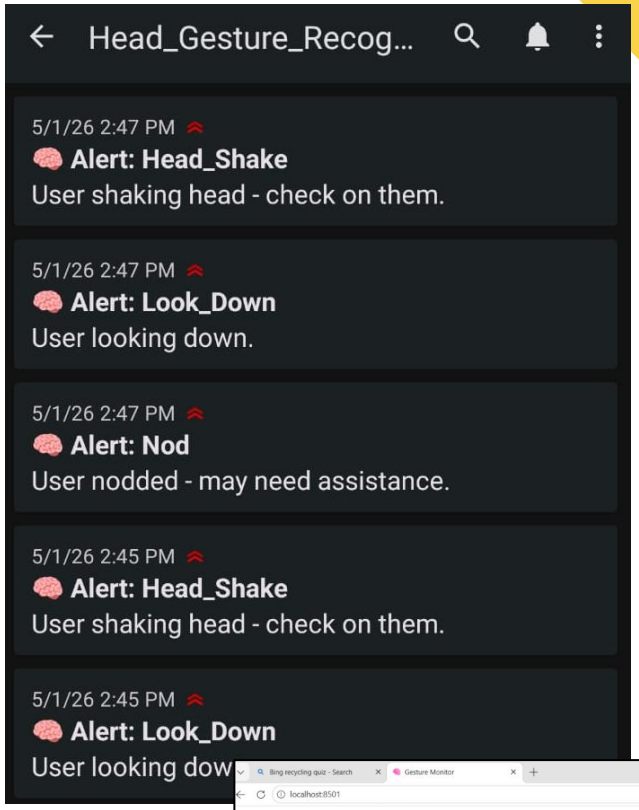
Streamlit Web Dashboard:

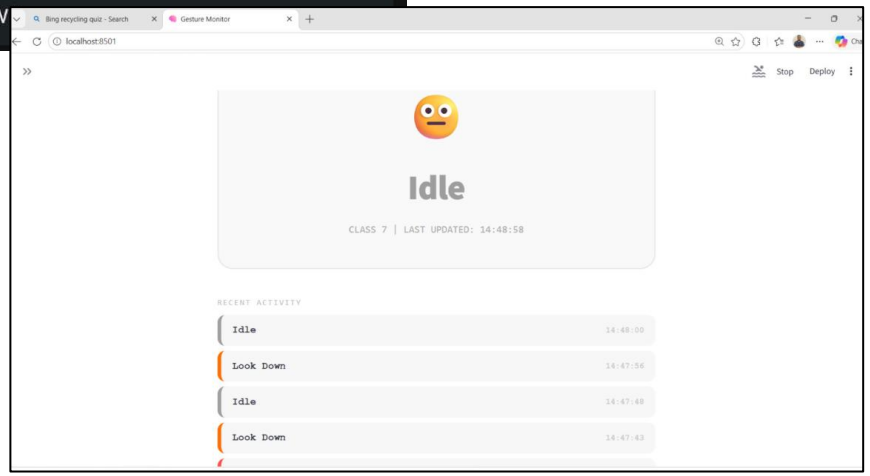
1. Provides caregivers with real-time visualization of the 12-axis motion data.

2. Logs gesture history to help medical professionals monitor patient activity over time.

Mobile App & Notifications:

A dedicated mobile application receives instant alerts and notifications for critical gestures.





Conclusion & Future Scope

Conclusion:

Edge-Native: 100% on-device inference for maximum privacy.

Dual-IMU Precision: Master-Slave setup eliminates motion ambiguity.

Full Ecosystem: Seamless integration (Hardware →Web→ Mobile).

Future Scope:

Model Pruning: Further optimize 1D-CNN for resource-constrained devices.

Low-Power Comms: Switch to BLE or wired UART to resolve heating issues and also to eliminate network dependency

Personalized AI: Use Transfer Learning to calibrate gestures for specific users.

Thank you

GitHub link:
abhaanisha/Head-Gesture-Recognition-System